FraudLens: detección de fraude en pagos digitales en tiempo real

Entregable 1: diagnóstico, requisitos y arquitectura

Dominio: fintech colombiana (billetera digital)

Andrés Felipe Vélez Álvarez (1021923619; afveleza@eafit.edu.co)

Samuel Samper Cardona (1040180930; ssamperc@eafit.edu.co)

Sebastián Salazar Henao (1022003377; salazarh3@eafit.edu.co)

Hellen Yanes Doria (1026137704; hyanesd@eafit.edu.co)

Universidad EAFIT

ST1630 Sistemas Intensivos en Datos

Profesor: Andrés Sacre Alzate

10 de octubre de 2026

FraudLens: detección de fraude en pagos digitales en tiempo real

1. Definición del problema

1.1 Contexto y problema de negocio

FraudLens es una billetera digital colombiana (proyecto académico, empresa ficticia) que procesa pagos con QR, transferencias inmediatas y pagos a comercios desde una app móvil. El equipo de riesgo detecta hoy el fraude con reportes que se calculan al cierre del día sobre la base de datos operacional. Cuando un analista identifica un patrón sospechoso, como una ráfaga de transferencias pequeñas hacia un mismo comercio recién creado o compras desde un dispositivo nunca visto en otra ciudad, el dinero ya salió y el cliente ya presentó el reclamo.

El problema es este: los analistas de riesgo de FraudLens necesitan detectar patrones de fraude en menos de 2 minutos desde que ocurre la transacción sospechosa, y además necesitan analizar el histórico completo para ajustar las reglas de detección con evidencia, sin afectar el rendimiento de la app de pagos.

1.2 Usuarios del sistema y decisiones que toman

La Tabla 1 resume quiénes usan el sistema y qué decisión toma cada uno con los datos.

Tabla 1
Usuarios del sistema y decisiones que toman con los datos

Usuario	Decisión que toma con los datos
Analista de fraude	Bloquear una cuenta, pedir verificación adicional o descartar una alerta, en los minutos siguientes al evento.
Jefe de riesgo	Ajustar umbrales de las reglas (montos, ventanas de tiempo) según la tasa de fraude por comercio, ciudad y franja horaria.
Equipo de comercios	Suspender o poner en revisión a comercios con montos sospechosos recurrentes.
Auditoría y cumplimiento	Verificar de dónde salió una cifra y reconstruir el estado de los datos en una fecha pasada.

1.3 Preguntas que el sistema debe responder

Operacionales (tiempo real)

- P1. ¿Qué clientes superan 5 transacciones o $2.000.000 COP acumulados en una ventana de 5 minutos?

- P2. ¿Qué transacciones vienen de un dispositivo nunca visto para ese cliente y con monto mayor a 3 veces su promedio de los últimos 30 días?
- P3. ¿Qué pagos de la última hora involucraron un comercio o un dispositivo de la lista de riesgo?

Analíticas (capa Gold)

- P4. ¿Cuál es la tasa de fraude (% de transacciones y % del monto) por comercio, ciudad y franja horaria, semana a semana?
- P5. ¿Cuál es la latencia promedio y percentil 95 entre la transacción y la generación de la alerta, y cuántas alertas dispara cada regla?
- P6. ¿Cuáles son los 10 comercios con mayor monto sospechoso en los últimos 7 días?

1.4 Por qué se necesita una arquitectura de datos distribuida

Con supuestos razonables para una billetera de tamaño medio, el problema tiene características que una base de datos relacional única no resuelve bien:

- Volumen: un promedio de 1.000 eventos por segundo equivale a unos 86 millones de eventos al día. Con cerca de 1 KB por evento en JSON son alrededor de 86 GB diarios y unos 30 TB al año.
- Velocidad: los picos llegan a 5.000 eventos por segundo (quincena, Día sin IVA, fin de año) y la detección exige cálculos con estado por cliente sobre ventanas de 5 minutos. Recalcularlos con consultas SQL sobre la tabla transaccional competiría con los pagos en curso.
- Variedad: llegan eventos JSON continuos, tablas relacionales de clientes y comercios, y archivos CSV de listas de riesgo. Cada fuente tiene formato, frecuencia y calidad distintos.
- Cargas mezcladas: la app necesita escrituras rápidas y consistentes (OLTP), mientras que los analistas necesitan agregaciones sobre meses de histórico (OLAP). Separarlas evita que una consulta pesada degrade los pagos.
- Reprocesamiento y auditoría: cuando cambia una regla hay que recalcular sobre datos históricos y poder reconstruir el estado de una fecha pasada, lo que exige almacenamiento inmutable y versionado (lakehouse) y no una tabla que se sobrescribe.

Por estas razones el sistema separa la ingestión de eventos (Kafka), el procesamiento escalable (Spark) y el almacenamiento en capas (Delta Lake), en lugar de concentrar todo en una sola instancia

de PostgreSQL (Kleppmann, 2017).

2. Requisitos funcionales y no funcionales

2.1 Requisitos funcionales

Cada requisito describe qué debe hacer el sistema, es verificable y se ubica en una de las cinco capas del diagrama de arquitectura (Sección 3). La columna "Verificación" de la Tabla 2 indica cómo se comprueba en la demo y qué pregunta de la Sección 1.3 responde.

Tabla 2

Requisitos funcionales

ID	Requisito funcional	Capa	Verificación
RF-01	El sistema debe ingerir en tiempo real los eventos de transacciones (pagos QR, transferencias inmediatas y pagos a comercios) publicándolos en el topic `transacciones` de Kafka, particionado por `id_cliente`, con latencia menor a 5 segundos desde su generación.	Ingestión	El productor simulado emite eventos; se mide el tiempo hasta que están disponibles en el topic.
RF-02	El sistema debe cargar periódicamente los catálogos de clientes y comercios desde la base de datos operacional (PostgreSQL 16) hacia la capa Bronze, con una frecuencia mínima diaria.	Fuente / Ingestión	El DAG `catalogos_diario` registra la carga; las tablas existen en Bronze con marca de tiempo.
RF-03	El sistema debe importar los archivos CSV de listas de riesgo (comercios y dispositivos en lista negra) a la capa Bronze cada vez que se publique una versión nueva.	Fuente / Ingestión	Al subir un CSV nuevo aparece una versión nueva en Bronze. Soporta P3.
RF-04	El sistema debe persistir los eventos crudos, sin modificarlos, en la capa Bronze (Delta Lake) junto con su offset de Kafka y su marca de tiempo de ingestión.	Almacenamiento (Bronze)	El número de `id_transaccion` distintos en Bronze coincide con los eventos únicos producidos (Bronze admite duplicados, ver RNF-03).
RF-05	El sistema debe validar, deduplicar por `id_transaccion` y estandarizar los eventos de Bronze hacia Silver. Los mensajes que no se pueden interpretar (JSON mal formado) van al topic `transacciones_dlq` y los que incumplen reglas de calidad (monto negativo, campos obligatorios vacíos) a una tabla de cuarentena en Silver con el motivo del rechazo.	Procesamiento / Silver	Se inyectan duplicados, JSON mal formado y registros inválidos; Silver solo contiene registros únicos y válidos, y cada rechazo aparece en la DLQ o en cuarentena.
RF-06	El sistema debe calcular con Spark Structured Streaming el número de transacciones y el monto acumulado por cliente en ventanas deslizantes de 5 minutos que avanzan cada minuto, y generar una alerta cuando se superen 5 transacciones o $2.000.000 COP.	Procesamiento (streaming)	Se envía una ráfaga de prueba y se verifica la alerta. Responde P1.

Continúa en la página siguiente

ID	Requisito funcional	Capa	Verificación
RF-07	El sistema debe generar una alerta cuando una transacción provenga de un dispositivo nunca visto para el cliente y su monto supere 3 veces su promedio de los últimos 30 días.	Procesamiento (streaming)	Caso de prueba con dispositivo nuevo y monto alto. Responde P2.
RF-08	El sistema debe generar una alerta cuando un pago tenga como destino un comercio de la lista de riesgo o provenga de un dispositivo en lista negra, cruzando el flujo de eventos con la versión vigente de la lista.	Procesamiento (join)	Pagos simulados a un comercio y desde un dispositivo de la lista disparan alerta. Responde P3.
RF-09	El sistema debe registrar cada alerta con su regla, transacción, cliente, comercio, marca de tiempo del evento y marca de tiempo de la alerta, en MongoDB (colección `alertas_fraude`) y en la tabla de alertas de Silver, sin registrar dos veces la misma alerta.	Almacenamiento / Consumo	La consulta por cliente y por regla devuelve los campos completos; tras reiniciar el job, el número de alertas no cambia.
RF-10	El sistema debe construir en Gold un modelo dimensional (tablas de hechos de transacciones y de alertas, y dimensiones de cliente, comercio, dispositivo, tiempo y regla) a partir de Silver.	Almacenamiento (Gold)	Una query con joins entre los hechos y las dimensiones se ejecuta correctamente.
RF-11	El sistema debe calcular en Gold la tasa de fraude (% de transacciones y % del monto) por comercio, ciudad y franja horaria, con corte semanal.	Procesamiento (batch) / Gold	Query analítica sobre Gold. Responde P4.
RF-12	El sistema debe calcular en Gold la latencia promedio y el percentil 95 entre la transacción y su alerta, y el número de alertas por regla.	Procesamiento (batch) / Gold	Query analítica sobre Gold. Responde P5.
RF-13	El sistema debe listar los 10 comercios con mayor monto sospechoso en los últimos 7 días.	Gold / Consumo	Query sobre Gold devuelve el top 10. Responde P6.
RF-14	El sistema debe permitir consultar la capa Gold con SQL mediante Trino (conector `delta_lake`) y mostrar dashboards de fraude por comercio, ciudad y hora en Metabase, sin acceder a la base de datos operacional.	Consumo	Un analista ejecuta P4–P6 en Trino y ve los resultados en el dashboard de Metabase.
RF-15	El sistema debe orquestar con Airflow todos los pasos batch en dos DAGs: `catalogos_diario` (PostgreSQL y CSV hacia Bronze y Silver, diario o al llegar un CSV nuevo) y `fraude_pipeline` (Silver $\rightarrow$ Gold, controles de calidad y actualización de Metabase, cada 10 minutos). Cada DAG debe detener los pasos posteriores cuando falle uno anterior, y un sensor debe verificar que el job de streaming esté activo.	Orquestación	La UI de Airflow muestra las dependencias; un fallo forzado detiene los pasos siguientes; si el streaming se detiene, el sensor lo reporta.
RF-16	El sistema debe permitir reprocesar un rango de fechas de Silver y Gold aplicando una versión nueva de las reglas (`rule_version`), sin modificar Bronze.	Procesamiento / Almacenamiento	Se reprocesa un día y se comparan resultados con la versión anterior.

Continúa en la página siguiente

ID	Requisito funcional	Capa	Verificación
RF-17	El sistema debe exponer las alertas mediante una API REST (FastAPI) consultable por cliente, regla y rango de tiempo, y notificar cada alerta nueva al equipo de fraude por webhook.	Consumo	Tras una ráfaga de prueba el webhook recibe la notificación y `GET /alertas` devuelve la alerta. Soporta P1–P3.

2.2 Requisitos no funcionales

Los valores de la Tabla 3 parten de los supuestos de la Sección 1.4 (1.000 eventos/s en promedio, picos de 5.000 eventos/s, $\sim$1 KB por evento) y de la meta de detección en menos de 2 minutos. La terminología de disponibilidad (CAP) y de garantías de entrega sigue a Kleppmann (2017), y el tratamiento de datos personales se rige por la Ley 1581 de 2012 (Congreso de la República de Colombia, 2012).

Tabla 3

Requisitos no funcionales

ID / Concepto	Requisito no funcional (valor concreto)	Cómo se verifica
RNF-01 Escalabilidad	El sistema debe soportar un pico sostenido de 5.000 eventos/s durante 30 minutos sin degradación (sin crecimiento continuo del lag del consumidor en Kafka). El topic tendrá 12 particiones para permitir hasta 12 tareas de consumo en paralelo, y el sistema debe poder escalar a 10.000 eventos/s agregando ejecutores de Spark sin cambios de código. Las particiones se dimensionan desde el inicio porque agregarlas después cambia qué partición recibe a cada `id_cliente` y rompe el orden por cliente durante la transición.	Prueba de carga con el productor a la máxima tasa que soporte el entorno local; se reportan el throughput medido y el lag del consumidor.
RNF-02 Disponibilidad (CAP)	Ingestión CP: Kafka con 3 brokers, `acks=all` y `min.insync.replicas=2`; si quedan menos de 2 réplicas sincronizadas rechaza escrituras en lugar de aceptar eventos que puedan perderse, porque un pago perdido no se recupera. Consulta de alertas con lectura AP: MongoDB (replica set de 3 nodos) escribe con `w=majority`, y las consultas de los analistas usan `readPreference secondaryPreferred`: si el primario cae siguen respondiendo desde los secundarios, aunque con segundos de retraso. Meta de disponibilidad: 99,5 % en ingestión y 99 % en consulta de alertas. En el despliegue local MinIO corre en un solo nodo y es punto único de falla; en producción se reemplaza por S3 o MinIO distribuido.	Se revisa la configuración; se detiene un broker y luego el primario de MongoDB, y se comprueba que la ingestión y la consulta de alertas siguen funcionando.

Continúa en la página siguiente

ID / Concepto	Requisito no funcional (valor concreto)	Cómo se verifica
RNF-03 Consistencia (garantía de entrega)	La ingestión a Kafka es at-least-once (`acks=all`, reintentos y productor idempotente). El paso Bronze → Silver → Gold logra efectivamente exactly-once con checkpoint de Spark en MinIO y escrituras idempotentes (MERGE por `id_transaccion`) en Delta Lake. Las alertas hacia MongoDB también son idempotentes: se escriben con upsert sobre un `_id` determinista (hash de `id_transaccion` o `id_cliente`, regla y ventana), así que reprocesar un micro-batch no duplica alertas. Se aceptan duplicados en Bronze, pero no en Silver, Gold ni en las alertas (duplicados = 0).	Se reinicia el job en mitad de un micro-batch y se comprueba que los conteos en Silver y en MongoDB no cambian.
RNF-04 Latencia	Latencia máxima desde la transacción hasta la alerta disponible: 30 s en el percentil 95 y 60 s en el máximo, cumpliendo la meta de negocio de 2 minutos. Para lograrlo el job usa un trigger de 5 s y la regla R1 emite con modo de salida `update` (en cada micro-batch, sin esperar el cierre de la ventana ni el watermark). Latencia máxima hasta que el evento esté en Gold: 15 minutos (`fraude_pipeline` corre cada 10 minutos). Las consultas analíticas sobre Gold deben responder en menos de 10 s para 30 días de datos.	Se mide con la marca de tiempo del evento frente a la de la alerta (RF-12).
RNF-05 Tolerancia a fallos	Spark corre en modo standalone (1 master y 2 workers). Si un worker o el driver fallan, el job se reinicia desde el último checkpoint y retoma en menos de 5 minutos sin pérdida de eventos (RPO = 0 para eventos ya en Kafka). Kafka replica cada partición en 3 réplicas sobre 3 brokers (tolera la caída de uno sin perder escrituras) y retiene los eventos 7 días, lo que permite reprocesar. Si falla un paso de un DAG, Airflow lo reintenta 3 veces con espera creciente y, si sigue fallando, detiene los pasos dependientes y notifica.	Se apaga un worker o un broker durante la demo y se mide el tiempo de recuperación.
RNF-06 Trazabilidad	Cada registro de Silver y Gold conserva el linaje hasta la fuente mediante columnas de auditoría (`source_system`, `kafka_topic`, `kafka_partition`, `kafka_offset`, `ingest_ts`, `batch_id`, `rule_version`). Delta Lake conserva 90 días de historial (`delta.logRetentionDuration` y `delta.deletedFileRetentionDuration` en 90 días) para reconstruir el estado de una fecha pasada con time travel. Toda cifra de Gold debe poder rastrearse hasta los eventos de Bronze que la generaron.	Se toma una cifra de Gold y se llega a sus eventos originales; se consulta una tabla con `VERSION AS OF`.
RNF-07 Seguridad y privacidad	Los datos personales (documento, teléfono, correo) se reemplazan desde Silver por un HMAC-SHA-256 con una clave secreta guardada fuera del lakehouse (variable de entorno o gestor de secretos), porque un hash simple de un documento o teléfono se puede adivinar por fuerza bruta; así Gold y los dashboards no los exponen; solo el rol de auditoría accede a Bronze con los originales. En el despliegue local no se configuran TLS ni cifrado en reposo; en producción se exigirían (TLS en Kafka y en las API, cifrado del almacenamiento de objetos) en línea con la Ley 1581 de 2012.	Se revisa que las consultas de analistas en Trino y Metabase no devuelvan datos personales en claro.

Continúa en la página siguiente

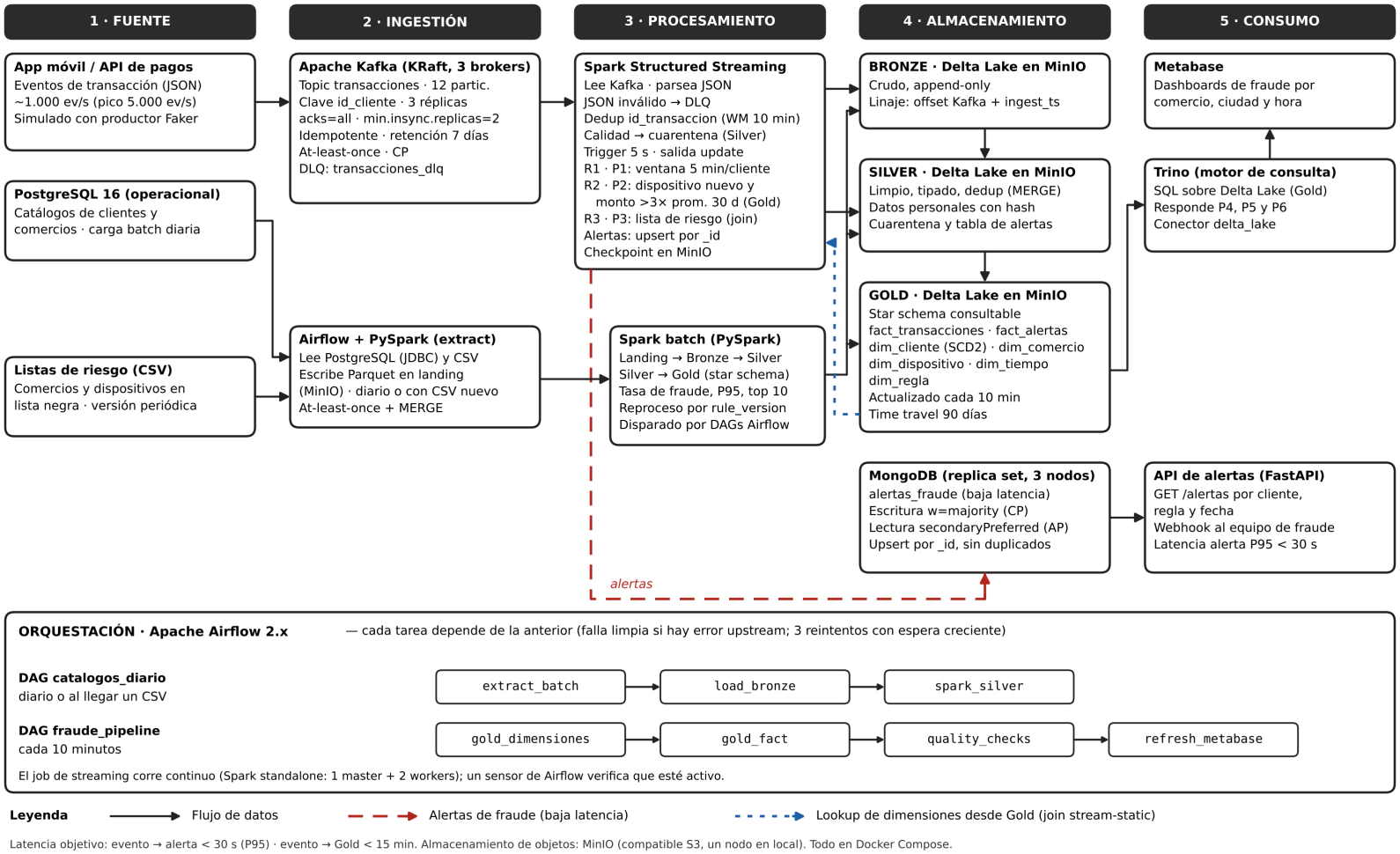
ID / Concepto	Requisito no funcional (valor concreto)	Cómo se verifica
RNF-08 Calidad de datos	Los registros que no cumplan el esquema o las reglas de calidad no deben detener el pipeline: van a la DLQ o a cuarentena con el motivo del rechazo, y la tasa de rechazo se reporta por ejecución (alerta si supera 1 %).	Se inyectan registros inválidos y se revisan la DLQ y la tabla de cuarentena.

3. Diagrama de arquitectura de datos por capas

La Figura 1 presenta el pipeline streaming y batch sobre un lakehouse Bronze/Silver/Gold, desplegado localmente con Docker Compose. Los archivos fuente están en el repositorio del grupo (`/docs/entregable1/arquitectura.drawio` y `arquitectura.png`).

Figura 1

Arquitectura de datos por capas de FraudLens



Nota. Enlace editable en Draw.io: abrir el diagrama en app.diagrams.net. Archivo fuente: `arquitectura.drawio`.

4. Justificación de decisiones técnicas (ADR)

Cada decisión no trivial se documenta con un Architecture Decision Record (ADR) de cuatro partes: contexto, alternativas consideradas, decisión y consecuencias. Los identificadores RF y RNF remiten a las Tablas 2 y 3, y los riesgos a la Tabla 6.

ADR-01: Apache Kafka con entrega at-least-once como capa de ingestión

Contexto. FraudLens recibe en promedio 1.000 eventos/s con picos de 5.000 (RNF-01) y debe alertar en menos de 2 minutos. Un pago perdido no se recupera, y cuando cambia una regla hay que reprocesar histórico sin volver a pedir los datos a la app (RF-16). Se necesita un registro durable, particionado y reproducible, desacoplado de la base operacional.
Alternativas consideradas. (a) *Escribir directo en PostgreSQL:* compite con los pagos en curso (carga OLTP) y no escala a 5.000 escrituras/s con ventanas calculadas encima. (b) *RabbitMQ:* es una cola que elimina el mensaje al consumirlo, por lo que no permite reprocesar ni que varios consumidores lean el mismo evento a su ritmo. (c) *Amazon Kinesis:* funcionalmente equivalente, pero ata el proyecto a AWS y el despliegue es local con Docker Compose.
Decisión. Kafka en modo KRaft con 3 brokers y un topic `transacciones` de 12 particiones, con clave `id_cliente`: todos los eventos de un cliente caen en la misma partición y conservan su orden, que es lo que necesitan las reglas con estado por cliente. Replicación 3, `acks=all`, `min.insync.replicas=2`, productor idempotente y retención de 7 días. Frente al teorema CAP, la ingestión es CP: ante una partición de red o la pérdida de réplicas sincronizadas, Kafka rechaza escrituras en lugar de aceptar eventos que podrían perderse. La garantía configurada es at-least-once; el efecto exactly-once se logra aguas abajo con MERGE por `id_transaccion` (RNF-03). Se descartó at-most-once porque pierde pagos, y las transacciones exactly-once de Kafka porque añaden complejidad que el MERGE idempotente ya resuelve.
Consecuencias. Se sacrifica disponibilidad de escritura si caen 2 de los 3 brokers. Los duplicados llegan a Bronze y deben eliminarse en Silver. Las 12 particiones no se pueden aumentar sin cambiar a qué partición va cada `id_cliente`, y por tanto sin romper el orden por cliente en la transición; por eso se dimensionan desde el inicio. Tres brokers consumen mucha memoria en un portátil (riesgo de la Tabla 6, mitigado con límites por contenedor).

ADR-02: Spark Structured Streaming frente a Flink o batch puro

Contexto. Las reglas R1 a R3 exigen cálculos con estado por cliente sobre ventanas deslizantes de 5 minutos, un cruce con las listas de riesgo y alertas con latencia P95 de 30 s (RNF-04). El mismo equipo debe además construir las capas Silver y Gold en batch (RF-10 a RF-13) y los cuatro integrantes deben poder defender todo el código.
Alternativas consideradas. (a) *Apache Flink:* procesa evento por evento con latencia de milisegundos y mejor manejo de estado, pero obliga a aprender y operar un segundo motor además de Spark, que el curso usa para el batch; la ganancia de latencia no se necesita con una meta de 30 s. (b) *Batch puro cada 10 minutos:* es simple, pero no cumple la meta de 2 minutos ni el requisito de streaming real, y la guía del curso no acepta simular con batch. (c) *Kafka Streams:* reduce piezas, pero está atado a la JVM y no ofrece la integración directa con Delta Lake ni con PySpark.
Decisión. Spark Structured Streaming con trigger de 5 s. R1 usa ventanas deslizantes de 5 minutos con avance de 1 minuto y modo de salida *update* dentro de `foreachBatch`: emite la alerta en cada micro-batch sin esperar el cierre de la ventana ni el watermark, que en modo *append* añadiría minutos

de retraso. Las alertas se escriben con upsert sobre un `_id` determinista, y el checkpoint en MinIO permite retomar tras una falla (RNF-03, RNF-05). Se usa un único motor para streaming y batch, con la misma API de DataFrames.
Consecuencias. Es micro-batch, no procesamiento por evento: la latencia mínima es de segundos, no de milisegundos. R2 y R3 hacen join stream-static con Gold, así que un dispositivo visto después de la última actualización puede marcarse como nuevo hasta 10 minutos (riesgo de la Tabla 6, aceptado y documentado). El estado de las ventanas y el watermark consumen memoria en los 2 workers. Si el volumen se multiplicara por 100, el primer cambio sería evaluar Flink para R1.

ADR-03: Delta Lake como lakehouse y modelo dimensional en Gold

Contexto. El sistema necesita almacenamiento inmutable y versionado para reprocesar y reconstruir el estado de una fecha pasada (RNF-06), escrituras idempotentes sin duplicados (RNF-03) y consultas analíticas sobre 30 días en menos de 10 s (RNF-04). Los analistas responden P4 a P6 con agregaciones por comercio, ciudad y franja horaria, y no deben tocar la base operacional.
Alternativas consideradas. (a) *Parquet sin formato de tabla:* no ofrece transacciones ACID ni MERGE, así que reprocesar o deduplicar obliga a reescribir carpetas completas. (b) *Apache Iceberg o Hudi:* son opciones válidas con prestaciones similares, pero la integración con Spark, el conector de Trino y la experiencia previa del equipo (laboratorios del curso) favorecen a Delta. (c) *PostgreSQL como almacén analítico:* mezcla cargas OLTP y OLAP y no escala a decenas de TB al año (Kleppmann, 2017).
Decisión. Delta Lake sobre MinIO (compatible con S3) con arquitectura Bronze, Silver y Gold. Delta aporta transacciones ACID, `MERGE` por `id_transaccion` y time travel con 90 días de historial. En Gold se usa un modelo dimensional (esquema en estrella): `fact_transacciones` y `fact_alertas` con las dimensiones `dim_cliente` (SCD tipo 2, para conservar el historial de atributos del cliente), `dim_comercio`, `dim_dispositivo`, `dim_tiempo` y `dim_regla`. El modelo en estrella es adecuado porque las preguntas P4 a P6 son agregaciones sobre hechos filtradas por dimensiones, y reduce los joins frente a un modelo normalizado.
Consecuencias. Trino necesita un metastore para ver las tablas Delta (riesgo de la Tabla 6, con Spark SQL como plan B). Hay que compactar archivos pequeños (`OPTIMIZE`) producidos por el streaming. Los cambios de esquema deben controlarse con el contrato de datos, porque un `mergeSchema` sin control rompe a los consumidores. MinIO en un solo nodo es punto único de falla en el despliegue local. El modelo dimensional exige diseñar las dimensiones por adelantado y es menos flexible para consultas no previstas.

ADR-04: MongoDB como almacén de alertas y su posición CAP

Contexto. Las alertas deben consultarse por cliente, regla y rango de tiempo con baja latencia (RF-09, RF-17), las escribe el streaming varias veces por segundo en picos y su estructura puede cambiar al agregar reglas. Los analistas deben poder consultarlas aunque falle un nodo.
Alternativas consideradas. (a) *PostgreSQL:* funciona, pero añade carga sobre una base de tipo OLTP y su esquema rígido encaja peor con alertas cuyos campos varían por regla. (b) *Cassandra:* AP por diseño y excelente para escrituras masivas, pero el volumen de alertas es mucho menor que el de eventos y su modelado por consultas es más costoso para un equipo de cuatro. (c) *Consultar solo Delta/Gold con Trino:* tiene latencia de segundos y de refresco de 10 minutos, insuficiente para la consulta operativa.
Decisión. MongoDB con replica set de 3 nodos, colección `alertas_fraude`, `_id` determinista

(hash de `id_transaccion` o `id_cliente`, regla y ventana) e índices por `id_cliente`, `regla` y `ts_alerta`. Posición CAP: MongoDB es por defecto CP, porque hay un único primario y, ante una partición, el lado sin mayoría no acepta escrituras. Las escrituras usan `w=majority` para no perder alertas confirmadas. Las lecturas usan `readPreference=secondaryPreferred`: si el primario cae, los secundarios siguen respondiendo, aunque con consistencia eventual (segundos de retraso). La elección es, por tanto, CP en escritura y disponibilidad priorizada en lectura, no que MongoDB sea AP en sí. Esto es aceptable porque una alerta con segundos de retraso no cambia la decisión del analista, mientras que perder una alerta sí.
Consecuencias. Durante una elección de primario (de segundos a decenas de segundos) no se pueden escribir alertas; el checkpoint de Spark y la idempotencia del `_id` permiten reintentar sin duplicados. Una lectura desde un secundario puede no mostrar la alerta más reciente. Se pierden joins y transacciones multidocumento complejas, lo cual no afecta a este caso. Tres nodos de MongoDB aumentan el consumo de memoria (se mitiga con caché reducida).

ADR-05: Apache Airflow y modelado de dependencias de los DAG

Contexto. Todos los pasos batch deben dispararse según dependencias, fallar de forma limpia si un paso anterior falla (RF-15) y dejar evidencia visible de su estado. Hay dos ritmos distintos: la carga de catálogos (diaria o al llegar un CSV nuevo) y la actualización de Gold (cada 10 minutos, para cumplir los 15 minutos de RNF-04).
Alternativas consideradas. (a) *Cron y scripts:* no modela dependencias, no reintenta con espera creciente y no ofrece interfaz para diagnosticar. (b) *Dagster o Prefect:* son alternativas modernas, pero Airflow es la referencia del curso y el equipo tiene más material para defenderla. (c) *Orquestar con Spark mismo:* mezcla orquestación con cómputo y no cubre extracción desde PostgreSQL ni la verificación del streaming.
Decisión. Airflow 2.x con LocalExecutor y dos DAG. `catalogos_diario`: `extract_batch` → `load_bronze` → `spark_silver`. `fraude_pipeline` (cada 10 minutos): `gold_dimensions` → `gold_fact` → `quality_checks` → `refresh_metabase`. Cada tarea depende de la anterior con la regla `all_success`, de modo que un fallo detiene los pasos siguientes en lugar de propagar datos corruptos. Cada tarea reintenta 3 veces con espera exponencial y, si sigue fallando, notifica. El job de streaming no es una tarea del DAG porque corre de forma continua; un sensor verifica que siga activo. Las tareas son idempotentes (MERGE), de modo que reejecutarlas no duplica datos.
Consecuencias. Airflow es un planificador por lotes, no un motor de streaming, por lo que Gold tiene una latencia de hasta 15 minutos. LocalExecutor ejecuta las tareas en una sola máquina y no escala horizontalmente; en producción se cambiaría a CeleryExecutor o KubernetesExecutor. El scheduler, el webserver y su base de metadatos añaden consumo de memoria. El pipeline lineal es simple pero no paraleliza las dimensiones, aspecto que se puede optimizar si el tiempo de ejecución se acerca a los 10 minutos.

5. Plan de implementación

El sistema se implementa entre S12 y S15 sobre un único docker-compose local. Cada integrante es dueño de un bloque del pipeline de extremo a extremo (código, pruebas y el ADR que lo sustenta) y es par revisor de otro bloque, para que los cuatro puedan defender cualquier parte del sistema en S16.

5.1 Distribución de responsabilidades

La Tabla 4 asigna a cada integrante los componentes y requisitos que implementa, junto con su par revisor.

Tabla 4

Distribución de responsabilidades entre los integrantes

Integrante	Componentes que implementa	Requisitos	Par revisor
Andrés Felipe Vélez Álvarez	*Ingestión e infraestructura.* `docker-compose.yml` (Kafka KRaft con 3 brokers, MinIO, PostgreSQL 16); topics `transacciones` (12 particiones, 3 réplicas) y `transacciones_dlq`; productor Python/Faker con escenarios de fraude, duplicados y registros malformados reproducibles (semilla fija); prueba de carga y medición del lag.	RF-01, RNF-01, RNF-02 y RNF-05 (Kafka)	Samuel Samper Cardona
Samuel Samper Cardona	*Procesamiento streaming.* Job de Spark Structured Streaming: Kafka → Bronze → Silver (dedup con watermark, DLQ, cuarentena, MERGE); reglas R1–R3 (ventanas de 5 min en modo update, joins stream-static con Gold); escritura idempotente de alertas en MongoDB y Silver; checkpoints en MinIO; medición de latencia P95.	RF-04 a RF-09, RNF-03, RNF-04, RNF-08	Sebastián Salazar Henao
Sebastián Salazar Henao	*Batch y Lakehouse.* Extracción de PostgreSQL (JDBC) y CSV a landing y Bronze; Silver de catálogos; Gold: `dim_cliente` (SCD2), `dim_comercio`, `dim_dispositivo`, `dim_tiempo`, `dim_regla`, `fact_transacciones` y `fact_alertas`; queries P4–P6; reproceso por `rule_version`; time travel de 90 días; hash de datos personales.	RF-02, RF-03, RF-10 a RF-13, RF-16, RNF-06, RNF-07	Hellen Yanes Doria
Hellen Yanes Doria	*Orquestación y consumo.* DAGs `catalogos_diario` y `fraude_pipeline` en Airflow (dependencias, reintentos, fallo limpio) y sensor del job de streaming; tarea `quality_checks`; MongoDB replica set con índices; API de alertas en FastAPI con webhook; Trino (conector `delta_lake`) y dashboards en Metabase; README.	RF-09 (consulta), RF-14, RF-15, RF-17, RNF-02 (MongoDB), RNF-05 (Airflow)	Andrés Felipe Vélez Álvarez

Acuerdos de trabajo: el contrato de datos (esquema JSON del evento y nombres de columnas) se fija en S12 y solo cambia por PR aprobado por los dueños afectados. Todo cambio entra a `main` por Pull

Request revisado por el par. Cada semana hay una sesión de 30 minutos en la que cada dueño explica su bloque a los demás. Cada integrante registra su uso de agentes de IA en `/docs/bitacora_ia.md`.

5.2 Cronograma e hitos

La Tabla 5 presenta el trabajo planeado por semana y el hito verificable que cierra cada una.

Tabla 5

Cronograma e hitos de implementación

Semana	Trabajo planeado	Hito verificable
S12 (actual)	Cierre y entrega del E1 (PDF, diagrama y PR). Contrato de datos; repositorio con carpetas por componente; matriz de versiones fija (Spark, Delta Lake, conectores); docker-compose con Kafka, MinIO, PostgreSQL, MongoDB y Airflow; productor publicando a Kafka.	H1: E1 entregado antes de las 11:59 p. m. del día anterior a S13; `docker compose up` levanta la base en los 4 equipos y el productor publica en el topic.
S13	Ajustes por la retroalimentación del E1. Streaming Kafka → Bronze → Silver con dedup, DLQ y cuarentena; catálogos y CSV a Bronze y Silver; prueba de concepto de Trino sobre Delta en MinIO; esqueleto de los DAGs.	H2: Silver tiene tantos `id_transaccion` únicos como eventos producidos; Trino consulta una tabla Delta.
S14	Reglas R1–R3 escribiendo en MongoDB; dimensiones y hechos de Gold; DAGs completos con dependencias y reintentos; API de alertas y webhook. Asesoría opcional con un problema documentado.	H3: primera alerta de extremo a extremo (evento → MongoDB → API) y P4 respondida con SQL sobre Gold.
S15	Dashboards en Metabase; reproceso por `rule_version`; pruebas de fallo (apagar un broker, matar el job de streaming, forzar un fallo en un DAG); prueba de carga y mediciones reales para las slides. Congelamiento de funcionalidades a mitad de semana; README, bitácora de IA, slides y simulacro de defensa cruzada.	H4: el pipeline corre desde cero en un equipo limpio siguiendo solo el README; P1–P6 respondidas; PR final listo.
Día antes de S16	Ensayo obligatorio de 25 minutos en el equipo de la demo; PR final abierto antes de las 11:59 p. m.	H5: la demo corre en vivo sin pasos manuales.

5.3 Riesgos técnicos y plan de mitigación

La Tabla 6 lista los riesgos técnicos identificados, su impacto, la mitigación prevista y quién responde por cada uno.

Tabla 6
Riesgos técnicos y plan de mitigación

Riesgo	Impacto	Mitigación	Responsable
El stack completo (3 brokers, Spark, Airflow, MongoDB x3, Trino, Metabase) no cabe en la RAM de un portátil.	Alto	Medir el consumo en S12; límites de memoria por contenedor, Airflow con LocalExecutor, MongoDB con caché reducida y perfiles de docker compose (núcleo / consumo). La demo corre en el equipo con más RAM.	Andrés Felipe Vélez Álvarez
La alerta de R1 supera 30 s si la ventana emite en modo append (espera el cierre de la ventana más el watermark).	Alto	Modo de salida update con `foreachBatch` y upsert en MongoDB; trigger de 5 s; medir el P95 desde S14.	Samuel Samper Cardona
Al reiniciar el job se reprocesa un micro-batch y se duplican alertas.	Medio	`_id` determinista por alerta con upsert (RNF-03); MERGE en Delta; prueba de reinicio forzado en S15.	Samuel Samper Cardona
Versiones incompatibles entre Spark, Delta Lake, conector de Kafka, hadoop-aws y Trino.	Alto	Matriz de versiones fija desde S12 e imagen Docker propia con los JARs; ninguna imagen con la etiqueta `latest`.	Andrés Felipe Vélez Álvarez
Trino no ve las tablas Delta en MinIO sin un metastore.	Medio	Prueba de concepto en S13 con Hive Metastore en contenedor o metastore de archivos (`register_table`). Plan B: responder P4–P6 con Spark SQL.	Hellen Yanes Doria
R2 marca como nuevos dispositivos ya vistos después de la última actualización de Gold (join stream-static).	Medio	`fraude_pipeline` refresca `dim_dispositivo` y el promedio de 30 días cada 10 minutos; se documenta como consecuencia en el ADR-02.	Sebastián Salazar Henao
El pipeline falla durante la demo en vivo.	Alto	Guion de demo con escenarios de fraude deterministas y dataset precargado; ensayo el día anterior; todos saben diagnosticar con la UI de Airflow, los logs de Spark y el lag de Kafka.	Todos
Un integrante no puede defender partes que no implementó (40 % del E2).	Alto	Par revisor por bloque, PRs revisados por otro integrante, explicación semanal y simulacro de preguntas en S15.	Todos

Referencias

Congreso de la República de Colombia. (2012, 17 de octubre). *Ley 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales*. Diario Oficial No. 48.587.

Kleppmann, M. (2017). *Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems*. O'Reilly Media.